

Kapitel 10

Der Präprozessor

10.1 Überblick

Den *Präprozessor*, den der Standard vorsieht, haben wir mit zwei seiner *Direktiven* bereits kennengelernt:

- Eine `include`-Direktive wird durch den Inhalt der in ihr genannten Datei ersetzt.
- Eine `define`-Direktive bewirkt nach unserem derzeitigen Kenntnisstand, dass der in ihr definierte Name im Rest des Moduls durch den ebenfalls in ihr definierten Ersatztext ersetzt wird.

Die grundsätzliche Arbeitsweise des Präprozessors kennen wir damit: Er überarbeitet den Text, indem er Einsetzungen, Ersetzungen – und unter Umständen auch Streichungen – vornimmt. Alles geschieht auf rein formaler Ebene, also ohne Rücksicht auf die logische Struktur der Funktionen, die der Modul enthält.

Man kann sich den Präprozessor als Programm vorstellen, das vor der eigentlichen Übersetzung läuft, oder auch als ersten Durchlauf des Compilers durch das Programm.

Was ein Standard-Präprozessor können muss, ist ebenso im Standard festgelegt wie die Form der Direktiven:

- Jede Direktive beginnt mit einem Nummernzeichen (`#`). Diese Nummernzeichen muss, von „white spaces“ abgesehen, das erste Zeichen in der Zeile sein.
- Dem Nummernzeichen folgt der Name der Direktive. Ob weitere Einträge erforderlich sind oder nicht, hängt von der jeweiligen Direktive ab.
- Jede Direktive wird durch ein Zeilenende-Zeichen beendet. Allerdings: Reicht der Platz in einer Zeile nicht aus, so kann man Fortsetzungszeilen schreiben. Dazu muss man *direkt* vor das Zeilenende-Zeichen der fortzusetzenden Zeile einen Backslash (`\`) setzen.

Wir haben für Direktiven also die allgemeine Form

```
#name [text]
```

Die Direktiven, die es neben `include` und `define` gibt, dienen im wesentlichen zur *bedingten Compilation*. Darunter versteht man, dass Teile des Quellcodes in Abhängigkeit von bestimmten Bedingungen übersetzt werden oder auch nicht. Bedingte Compilation ist in zwei Fällen besonders nützlich:

- Sie erlaubt es, Headerdateien so zu formulieren, dass man in beliebiger Reihenfolge auf sie zugreifen kann.

- Varianten von Funktionen, die sich nur in Details unterscheiden, können in einer Quelldatei gespeichert werden. Die Entscheidung darüber, welche Variante übersetzt werden soll, braucht erst beim Aufruf des Compilers getroffen zu werden.

10.2 Die Direktive `#include`

Die `include`-Direktive kennen wir bereits vollständig. Ihre beiden Formen sind

```
#include <name>
#include "name"
```

Zur Erinnerung noch einmal der Unterschied zwischen den beiden Formen: Bei der zweiten Form sucht der Präprozessor die angegebene Datei zunächst im aktuellen Verzeichnis. Findet er sie dort nicht, sucht er in anderen Directories, die implementationsspezifisch festgelegt sein müssen. Die erste Form arbeitet ebenso, nur dass die Suche in der aktuellen Directory entfällt.

Faktisch heißt das:

- die spitzen Klammern werden bei Standard-Headerdatei verwendet,
- die Gänsefüßchen bei eigenen Dateien.

Der Name darf jeweils auch Pfadangaben enthalten.

10.3 Makros (`#define`)

Dass man mit der Direktive `#define` Konstanten benennen kann, haben wir bereits gelernt und genutzt. Tatsächlich ist diese Möglichkeit nur ein Teilaspekt der Direktive. Zur Erinnerung noch einmal die Form der Direktive:

```
#define name [ersatztext]
```

An dem, was wir über den Eintrag *name* bereits wissen, ändert sich zunächst nur die Nomenklatur: Namen, die in einer `define`-Direktive definiert werden, werden vielfach als *Makros* bezeichnet. Die Ersetzungen, die ein Makro bewirkt, bezeichnet man als *Expansion des Makro*.

Als *ersatztext* sind, über unsere bisherigen Kenntnisse hinaus, beliebige Zeichenfolgen erlaubt. Diese Zeichenfolgen dürfen insbesondere auch bereits zuvor definierte Makros enthalten oder leer sein.

Zulässig sind so zum Beispiel

```
#define BREITE 10
#define HOEHE (BREITE + 5)
```

Makros können Parameter erhalten:

```
#define name(parameter) ersatztext
```

Wichtig ist, dass die öffnende Klammer dem Namen *unmittelbar* folgen muss – damit wird gekennzeichnet, dass die Klammer eine Parameterliste einleitet und nicht Bestandteil des Ersatztextes ist. Zum Beispiel können wir also

```
#define QUADRAT(x) x * x
```

definieren. An den Stellen, an denen der Makro expandiert werden soll, müssen Argumente angegeben werden, ähnlich wie bei einer Funktion. Die Folge ist praktisch eine *doppelte* Textersetzung: Der Makro einschließlich seiner Argumentliste wird durch den Ersatztext ersetzt; im Ersatztext werden die Parameter durch die Argumente ersetzt. So ergeben zum Beispiel die Aufrufe

```
x = QUADRAT(7.4);  
y = QUADRAT(x + 1);
```

letztlich die Quellenweisungen

```
x = 7.4 * 7.4;  
y = x + 1 * x + 1;
```

Das zweite Beispiel zeigt, dass der Makro `QUADRAT` *unvollständig* definiert ist: Für `y` resultiert ein Ausdruck, der offensichtlich *nicht* der Intention entspricht. Abhilfe schafft

```
#define QUADRAT(x) ((x) * (x))
```

Die äußeren Klammern sind nötig, da es Operatoren gibt, deren Priorität höher als die der Multiplikation ist.

Makros mit Parametern können wie Funktionen die Lesbarkeit eines Programms erhöhen und Schreibarbeit sparen. Man muss sich aber darüber klar sein, dass zwischen Makros und Funktionen ganz wesentliche Unterschiede bestehen, die letztlich alle aus der Tatsache resultieren, dass Makros expandiert werden, *bevor* die eigentliche Übersetzung beginnt und dass sie damit im ausführbaren Programm überhaupt nicht mehr in Erscheinung treten. Einige Details:

- Dass auch bei Makros die Anzahlen von Parametern und Argumenten übereinstimmen müssen, sollte klar sein. Die Parameter von Makros besitzen jedoch *keine* Typen, so dass die Argumente beliebige Typen besitzen können – Typen spielen ja erst später eine Rolle, wenn der Compiler übersetzt.
- Jeder Aufruf eines Makro wird durch seinen Ersatztext ersetzt. Das hat einerseits zur Folge, dass der Maschinencode, der aus dem Ersatztext resultiert, mehrfach im ausführbaren Programm steht, während der Maschinencode, der aus einer Funktion resultiert, nur einmal im Programm steht. Dadurch entfallen andererseits die Sprünge und die Parameterzuordnung, mit denen Funktionsaufrufe verbunden sind.

Aus der Tatsache, dass ein Makro-Aufruf durch Textersetzung aufgelöst wird, folgt auch, dass man vorsichtig sein muss, wenn man als Argumente Ausdrücke mit Nebeneffekten einsetzt. Aus dem Aufruf

```
i = QUADRAT(j++);
```

resultiert so

```
i = ((j++) * (j++));
```

Abgesehen davon, dass der doppelte Nebeneffekt in der Regel nicht beabsichtigt sein wird, ist er auch gefährlich. Mit den Gefahren von Nebeneffekten haben wir uns bereits in Abschnitt 4.11 befasst.

Man kann die Argumente von einem Makro auch als Zeichenkette verwenden. Dazu muss man ihnen bei der Verwendung `#` voranstellen. Ein nützliches Beispiel ist im Abschnitt 10.5 angegeben.

10.4 Bedingte Compilation (`#if`, `#elif`, `#else`)

Die Direktiven-Konstrukte, die bedingte Compilation erlauben, arbeiten mit der Logik der `if`-Anweisungen. Ihre Form ist allerdings durchaus anders:

```
#if bedingung  
    quellecode
```

```
[ #elif bedingung
    quellcode ]
...
[ #else
    quellcode ]
#endif
```

Die Bedingungen müssen konstante Ausdrücke sein und werden in der C-typischen Weise als *wahr* oder *falsch* interpretiert. Der Quellcode, der für die einzelnen Fälle vorgesehen ist, kann aus vollständigen Anweisungen bestehen, braucht es aber nicht. Selbstverständlich muss der Programmierer immer dafür sorgen, dass der Quellcode, der durch die Auswahl entsteht, vollständige Quellanweisungen ergibt.

Wir betrachten ein Beispiel: Wir wollen einen Modul testen und dazu Ausgabeanweisungen in den Quellcode einfügen. Das können wir etwa in dieser Form tun:

```
#define TESTEN 1
...
#if TESTEN
    printf ("Kontrollpunkt A erreicht\n")
#endif
```

Haben wir unsere Tests abgeschlossen, so brauchen wir die `define`-Direktive für `TESTEN` nur durch

```
#define TESTEN 0
```

zu ersetzen, während alle übrigen Testzusätze im Modul stehen bleiben können: Der Präprozessor entfernt sie, so dass sie in Zukunft nicht mit übersetzt werden. Der wesentliche Vorteil dieses Verfahrens zeigt sich, wenn wir Änderungen am Modul vornehmen und ihn danach erneut testen müssen: Durch erneute Änderung der `define`-Direktive können wir alle Testausgabe wieder aktivieren.

In einem Punkt ist das Beispiel noch ziemlich untypisch: Man wird dem Namen `TESTEN` keinen Wert geben, sondern nur

```
#define TESTEN
```

schreiben. Der Name wird dadurch zwar definiert, repräsentiert jedoch keinen Wert. Abfragen kann man das durch den Präprozessor-Operator

```
defined (name)
```

Er prüft nur, ob der Name *name* definiert ist oder nicht; welchen Wert der Name ggf. repräsentiert, wird *nicht* geprüft. Wohlgemerkt: Dieser Operator steht nur in Präprozessor-Direktiven zur Verfügung. Unser Beispiel könnte damit so aussehen:

```
#define TESTEN
...
#if defined (TESTEN)
    printf ("Kontrollpunkt A erreicht\n")
#endif
```

Dieses kann man mit einer weiteren Direktive noch kürzer und prägnanter schreiben: Die Direktive

```
#ifdef name
```

hat gerade dieselbe Wirkung wie

```
#if defined (name)
```

Bei `#ifdef` kann immer nur ein Name angegeben werden; bei `if` könnten wir zum Beispiel aber auch

```
#if defined (name1) || defined (name2)
```

schreiben. `ifdef` ist also tatsächlich nur eine Kurzform für Spezialfälle.

Es sollte jetzt auch schon klar sein, wie man Größen in mehreren Headerdateien definieren kann, ohne dass es zu Konflikten kommt, unabhängig von der Reihenfolge, in der auf die Headerdateien zugegriffen wird. Schreiben wir etwa

```
#if !defined (EOF)
    #define EOF (- 1)
#endif
```

so ist `EOF` beim ersten Auftreten dieser Direktivenfolge noch nicht definiert – die `define`-Direktive wird also wirksam. Findet der Präprozessor dieselbe Direktivenfolge erneut, ist `EOF` bereits definiert, so dass er die Wiederholung aus dem zu übersetzenden Quellcode eliminiert.

10.5 Weitere Möglichkeiten (`#ifndef`, `#undef`)

Neben den Direktiven, die wir kennengelernt haben, schreibt der Standard vier weitere Direktiven vor:

- `ifndef`** ist das Pendant zu `ifdef`: Geprüft wird, ob der angegebene Name *nicht* definiert ist.
- `undef`** streicht den angegebenen Namen aus der Liste der definierten Namen, löscht also praktisch den Makro.
- `error`** erlaubt die Ausgabe eigener Fehlermeldungen während der Compilation.
- `line`** erlaubt die Einflussnahme auf die Form der Meldungen des Compilers.

Daneben schreibt der Standard die Vordefinition von fünf Makros vor:

- `__FILE__`** Name der Quelldatei, die gerade übersetzt wird
- `__LINE__`** Nummer der Quellzeile, die gerade übersetzt wird
- `__DATE__`** Datum der Übersetzung
- `__TIME__`** Uhrzeit der Übersetzung
- `__STDC__`** *wahr* oder *falsch*, je nachdem, ob der Compiler ein Standardcompiler ist oder nicht

Diese Makros sind faktisch Schlüsselworten gleichgestellt und dürfen vom Programmierer nicht für eigene Zwecke verwendet werden.

Für die Fehlersuche in großen Projekten mit mehreren Quelldateien können diese Makros sehr nützlich sein. Zum Beispiel liefert

```
#define DEBUG
...
#ifdef DEBUG
#define REPORT printf("Zeile %d in %s erreicht\n", \
    __LINE__, __FILE__)
#else
#define REPORT
#endif
```

eine einfache und ausbaufähige Hilfe bei der Überwachung des Programmablaufs. Manchmal will man sich bei der Ausführung eines Programms davon überzeugen, dass eine bestimmte Programmzeile auch erreicht wurde. An solchen kritischen Stellen fügt man einfach die Zeile `REPORT`; in den Quelltext ein und erhält dann eine aussagekräftige Kontrollausgabe, sofern `DEBUG` bei der Compilation definiert wurde. Ausbaubar ist das Makro z.B. so:

```
#define REPORTINT(x) printf("%s(%d) : Wert: %d\n", \
                          __FILE__, __LINE__, x)
```

Damit kann der aktuelle Wert von `int`-Variablen ausgegeben werden. Auf `double`-Variablen angewendet liefert das Makro natürlich Unsinn.

Schließlich kann man durch folgende Variante auch noch dafür sorgen, dass der Name der angesprochenen Variablen mit ausgegeben wird.

```
#define REPORTINT(x) printf("%s(%d) : " #x " == %d\n", \
                          __FILE__, __LINE__, x)
```

Wie schon erwähnt sorgt `#` dafür, dass das Argument in Anführungszeichen eingeschlossen wird. `REPORTINT(a)`; wird also zu

```
printf("%s(%d) : " "a" " == %d\n", ..., ..., a);
```

expandiert. Durch die Zusammensetzung von Zeichenkettenkonstanten ergibt sich ein zusammenhängender Formatstring für `printf`. Maßnahmen wie solche raffinierten Testausgaben fass man untr dem Begriff „low-level debugging“ zusammen. Geschickt eingesetzt können sie Programmierer sehr bei der Fehlersuche unterstützen.

10.6 Makro-Definition im Compileraufruf

Wir haben gesehen, wie man Makros mit den Direktive `define` definieren bzw. mit der Direktive `undef` löschen kann. Beides kann man auch durch Optionen im Aufruf des Compilers erreichen. Der `gcc`-Compiler unterstützt dazu die Kommandozeilenparameter:

```
-Dname
-Dname=text
-Uname
```

Im Zusammenhang mit bedingter Compilation wird man diese Möglichkeit häufig nutzen, weil sie es einem erspart, zwischen den verschiedenen Compilerläufen den Quellcode zu modifizieren. Man kann zum Beispiel direkt im Compileraufruf angeben, ob Testausgaben erzeugt werden soll oder nicht, ohne den entsprechenden Makro im Quellcode zu definieren oder zu löschen.